

Laboratorio3_AntonioVinicius

September 23, 2026

1 LAB 3 - MÉTODOS NUMÉRICOS - ANTÔNIO VINÍCIUS LIBERATO OLIVEIRA SIEBRA - 610619

```
[1]: import numpy as np
import pandas as pd
from matplotlib.pyplot import subplots
import statsmodels.api as sm
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split
from ISLP.models import (ModelSpec as MS, summarize)
```

Importação das bibliotecas essenciais para manipulação de dados, visualização, modelagem estatística (statsmodels) e machine learning (sklearn).

```
[2]: df = pd.read_csv('dataset_regressao_limpo.csv')
df.columns = df.columns.str.strip() # Remove espaços extras nos nomes das
    ↪colunas
df.head()
```

```
[2]:
```

	AMBIENT_TEMPERATURE	MODULE_TEMPERATURE	IRRADIATION	DC_POWER
0	25.184316	22.857507	0.0	0.0
1	25.184316	22.857507	0.0	0.0
2	25.184316	22.857507	0.0	0.0
3	25.184316	22.857507	0.0	0.0
4	25.184316	22.857507	0.0	0.0

Carregamento da base dataset_regressao_limpo.csv e visualização das primeiras linhas.

Transformação da Variável Alvo: Como a variável DC_POWER é contínua, precisamos transformá-la em categórica para aplicar métodos de classificação. Vamos utilizar a **mediana** como ponto de corte. Valores acima da mediana serão classificados como **Positivo** (alta geração) e valores iguais ou abaixo serão **Negativo** (baixa/nenhuma geração).

```
[3]: mediana_power = df['DC_POWER'].median()
df['CLASS_POWER'] = np.where(df['DC_POWER'] > mediana_power, 'Positivo',
    ↪'Negativo')
print("Mediana do DC_POWER:", mediana_power)
print(df['CLASS_POWER'].value_counts())
```

Mediana do DC_POWER: 428.57142855

CLASS_POWER

Negativo 34387

Positivo 34387

Name: count, dtype: int64

Cálculo da matriz de correlação para as variáveis numéricas do dataset, permitindo checar as relações entre as temperaturas, irradiação e a potência gerada.

```
[4]: df[['AMBIENT_TEMPERATURE', 'MODULE_TEMPERATURE', 'IRRADIATION', 'DC_POWER']].  
      ↪corr()
```

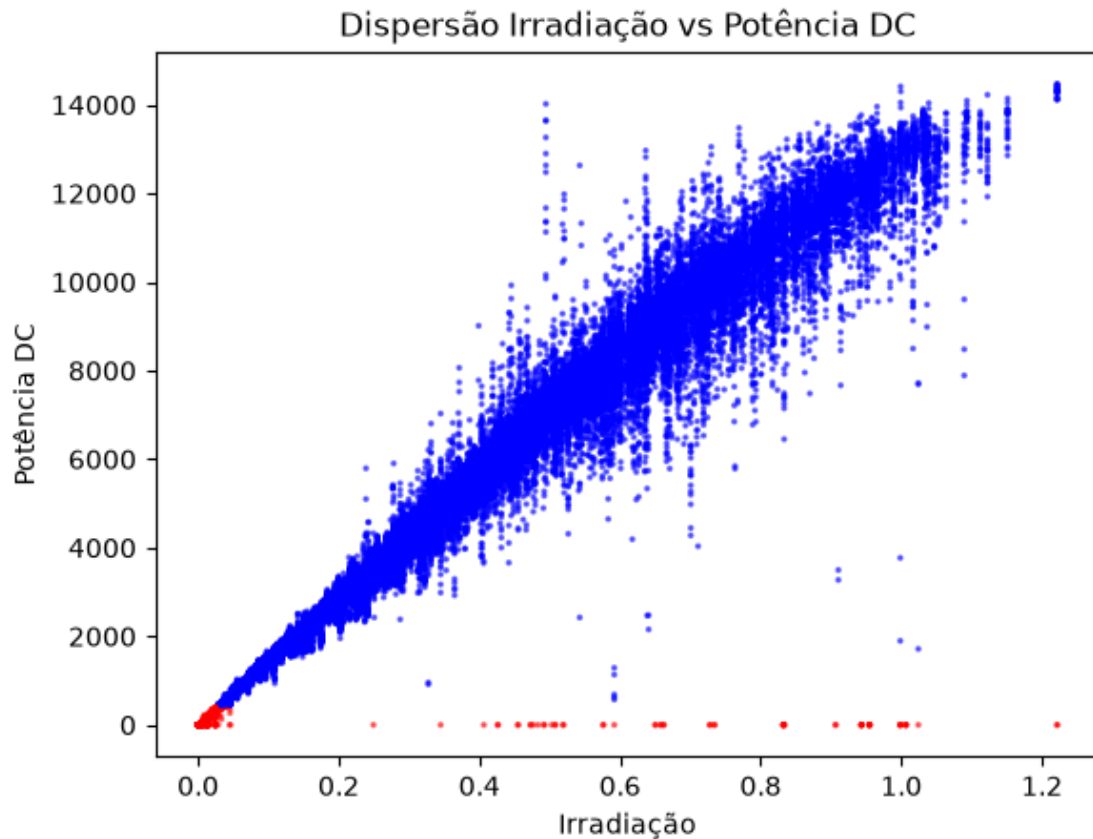
```
[4]:
```

	AMBIENT_TEMPERATURE	MODULE_TEMPERATURE	IRRADIATION	\
AMBIENT_TEMPERATURE	1.000000	0.855420	0.725278	
MODULE_TEMPERATURE	0.855420	1.000000	0.961511	
IRRADIATION	0.725278	0.961511	1.000000	
DC_POWER	0.724668	0.954811	0.989391	

	DC_POWER
AMBIENT_TEMPERATURE	0.724668
MODULE_TEMPERATURE	0.954811
IRRADIATION	0.989391
DC_POWER	1.000000

Gráfico de dispersão entre a Irradiação e a Potência DC, colorido pelas classes recém-criadas.

```
[5]: import matplotlib.pyplot as plt  
      colors = {'Positivo': 'blue', 'Negativo': 'red'}  
      plt.scatter(df['IRRADIATION'], df['DC_POWER'], c=df['CLASS_POWER'].map(colors),  
                  ↪alpha=0.5, s=2)  
      plt.xlabel('Irradiação')  
      plt.ylabel('Potência DC')  
      plt.title('Dispersão Irradiação vs Potência DC')  
      plt.show()
```



Configuração da matriz de modelo com os preditores (Temperaturas e Irradiação). Em seguida, treina-se um Modelo Linear Generalizado (GLM) para Regressão Logística (Binomial) a fim de prever se a geração de energia será **Positiva**.

```
[6]: preditores = ['AMBIENT_TEMPERATURE', 'MODULE_TEMPERATURE', 'IRRADIATION']
design = MS(preditores)
X = design.fit_transform(df)
y = df['CLASS_POWER'] == 'Positivo'

glm = sm.GLM(y, X, family=sm.families.Binomial())
results = glm.fit()
summarize(results)
```

```
[6]:
```

	coef	std err	z	P> z
intercept	-6.3914	0.265	-24.137	0.000
AMBIENT_TEMPERATURE	-0.0933	0.038	-2.467	0.014
MODULE_TEMPERATURE	0.2252	0.034	6.622	0.000
IRRADIATION	68.5074	1.435	47.725	0.000

Extraí as probabilidades previstas pelo modelo logístico. Classifica como **Positivo** as instâncias com probabilidade > 50% e monta uma matriz de confusão para avaliar a acurácia no próprio dado

de treino.

```
[7]: probs = results.predict()
labels = np.array(['Negativo'] * len(df))
labels[probs > 0.5] = 'Positivo'

confusion_matrix(df['CLASS_POWER'], labels)
```

```
[7]: array([[34323,    64],
          [ 903, 33484]])
```

Cria uma divisão real de treino e teste. O modelo será treinado com 70% dos dados e validado com os 30% restantes para evitar *overfitting* e obter uma taxa de erro realista.

```
[8]: X_train, X_test, y_train_cat, y_test_cat = train_test_split(X,
    ↪df['CLASS_POWER'], test_size=0.3, random_state=42)
y_train = y_train_cat == 'Positivo'
y_test = y_test_cat == 'Positivo'
X_test.shape
```

```
[8]: (20633, 4)
```

Reajusta a regressão logística usando apenas os dados de treino e calcula a matriz de confusão exclusivamente com os dados da base de teste.

```
[9]: glm_train = sm.GLM(y_train, X_train, family=sm.families.Binomial())
results_train = glm_train.fit()
probs_test = results_train.predict(exog=X_test)

labels_test = np.array(['Negativo'] * len(X_test))
labels_test[probs_test > 0.5] = 'Positivo'
confusion_matrix(y_test_cat, labels_test)
```

```
[9]: array([[10361,    13],
          [ 329, 9930]])
```

Prepara os dados para a biblioteca `scikit-learn` (removendo a coluna de intercepto) e ajusta o modelo de Análise Discriminante Linear (LDA). Exibe as médias das variáveis para cada classe.

```
[10]: X_train_drop = X_train.drop(columns=['intercept'])
X_test_drop = X_test.drop(columns=['intercept'])

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
lda = LDA(store_covariance=True)
lda.fit(X_train_drop, y_train_cat)
lda.means_
```

```
[10]: array([[2.33084026e+01, 2.15933938e+01, 2.68214130e-03],
          [2.78012392e+01, 4.09024993e+01, 4.62436547e-01]])
```

Gera predições usando o modelo LDA no conjunto de teste e exibe a matriz de confusão.

```
[11]: lda_pred = lda.predict(X_test_drop)
      confusion_matrix(y_test_cat, lda_pred)
```

```
[11]: array([[10361,    13],
            [ 2326,   7933]])
```

Ajusta e prevê usando Análise Discriminante Quadrática (QDA). Este modelo lida com covariâncias separadas para cada classe, sendo mais flexível do que o LDA.

```
[12]: from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis as QDA
      qda = QDA(store_covariance=True)
      qda.fit(X_train_drop, y_train_cat)
      qda_pred = qda.predict(X_test_drop)
      confusion_matrix(y_test_cat, qda_pred)
```

```
[12]: array([[10265,   109],
            [  710,  9549]])
```

Ajusta um modelo Naive Bayes, que assume que todos os preditores são independentes entre si dentro de cada classe, e exibe a performance via matriz de confusão.

```
[13]: from sklearn.naive_bayes import GaussianNB
      nb = GaussianNB()
      nb.fit(X_train_drop, y_train_cat)
      nb_pred = nb.predict(X_test_drop)
      confusion_matrix(y_test_cat, nb_pred)
```

```
[13]: array([[10147,   227],
            [  853,  9406]])
```

Aplica o algoritmo de K-Vizinhos Mais Próximos (KNN) com K=1. Como é um modelo não-paramétrico, a predição é baseada puramente na observação mais próxima da amostra.

Nota: O KNN requer padronização dos dados para funcionar bem, pois é baseado em distâncias. Faremos isso aqui com StandardScaler.

```
[14]: from sklearn.preprocessing import StandardScaler
      from sklearn.neighbors import KNeighborsClassifier as KNN

      scaler = StandardScaler()
      X_train_std = scaler.fit_transform(X_train_drop)
      X_test_std = scaler.transform(X_test_drop)

      knn1 = KNN(n_neighbors=1)
      knn1.fit(X_train_std, y_train_cat)
      knn1_pred = knn1.predict(X_test_std)
      confusion_matrix(y_test_cat, knn1_pred)
```

```
[14]: array([[10348,    26],
           [    54, 10205]])
```

Refaz o KNN com $K=3$, reduzindo a variabilidade extrema ao usar o consenso dos 3 vizinhos mais próximos para a classificação.

```
[15]: knn3 = KNN(n_neighbors=3)
      knn3.fit(X_train_std, y_train_cat)
      knn3_pred = knn3.predict(X_test_std)
      confusion_matrix(y_test_cat, knn3_pred)
```

```
[15]: array([[10348,    26],
           [    26, 10233]])
```

O laboratório avança testando o KNN com $K = 3$ e $K = 5$ para verificar se o aumento de vizinhos melhora a precisão ao identificar instâncias da classe **Positivo** (alta geração de energia).

```
[16]: knn3 = KNN(n_neighbors=3)
      knn3.fit(X_train_std, y_train_cat)
      knn3_pred = knn3.predict(X_test_std)
      print("Erro com K=3:", np.mean(y_test_cat != knn3_pred))
      print(confusion_matrix(y_test_cat, knn3_pred))

      knn5 = KNN(n_neighbors=5)
      knn5.fit(X_train_std, y_train_cat)
      knn5_pred = knn5.predict(X_test_std)
      print("\nErro com K=5:", np.mean(y_test_cat != knn5_pred))
      print(confusion_matrix(y_test_cat, knn5_pred))
```

```
Erro com K=3: 0.002520234575679736
[[10348    26]
 [    26 10233]]
```

```
Erro com K=5: 0.002180972228953618
[[10341    33]
 [    12 10247]]
```

Por fim, ajusta-se um modelo de Regressão Logística alterando o limiar de decisão. Na nossa base a proporção original é balanceada, mas testar um limiar diferente (como 75% em vez do padrão 50%) permite que o modelo seja mais rigoroso (conservador) na hora de classificar a geração como **Positiva**, reduzindo falsos positivos.

```
[17]: # O statsmodels GLM precisa da adição manual de uma constante (intercepto)
      X_train_log = sm.add_constant(X_train_drop)
      X_test_log = sm.add_constant(X_test_drop)

      # Convertendo a classe 'Positivo'/'Negativo' para 1/0 para a regressão logística
      y_train_bin = (y_train_cat == 'Positivo').astype(int)
```

```

glm_custom = sm.GLM(y_train_bin, X_train_log, family=sm.families.Binomial())
results_custom = glm_custom.fit()

probs_custom = results_custom.predict(exog=X_test_log)

# Aplicando o novo limiar de decisão de 75%
labels_custom = np.array(['Negativo'] * len(X_test_log))
labels_custom[probs_custom > 0.75] = 'Positivo'

confusion_matrix(y_test_cat, labels_custom)

```

```

[17]: array([[10361,   13],
           [ 597,  9662]])

```

1.0.1 4.7.7 Regressão Linear e de Poisson nos Dados Contínuos

A última seção aborda a modelagem direta da variável contínua original. O objetivo agora é voltar a prever a potência contínua gerada (DC_POWER), comparando as limitações da Regressão Linear com a adequação da Regressão de Poisson (adequada para dados estritamente não-negativos).

```

[18]: Y_cont = df['DC_POWER'].astype(int) # O modelo de Poisson espera números
      ↪ inteiros/contagens

# Divisão sequencial em treino (primeiras 50000 observações) e teste (restante)
X_train_cont, X_test_cont = X[:50000], X[50000:]
y_train_cont, y_test_cont = Y_cont[:50000], Y_cont[50000:]

```

Ajusta um modelo de Regressão Linear padrão (OLS). Apesar de produzir resultados, a regressão linear permite teoricamente prever valores negativos de potência, o que não faz sentido físico para o painel solar (que no mínimo gera 0 à noite).

```

[19]: lin_mod = sm.OLS(y_train_cont, X_train_cont).fit()
      summarize(lin_mod)

```

```

[19]:
           coef  std err          t  P>|t|
intercept    -289.5827    27.098   -10.687  0.000
AMBIENT_TEMPERATURE    -4.3519     1.945    -2.238  0.025
MODULE_TEMPERATURE     20.9522     1.365    15.355  0.000
IRRADIATION    12330.0000    42.875   287.498  0.000

```

Ajusta um Modelo Linear Generalizado (GLM) utilizando a família de distribuição de Poisson. Esta abordagem modela diretamente assumindo uma função de ligação logarítmica (log-link). Isso garante que as previsões do modelo sejam sempre valores não-negativos (positivos ou zero).

```

[20]: pois_mod = sm.GLM(y_train_cont, X_train_cont, family=sm.families.Poisson()).
      ↪ fit()
      summarize(pois_mod)

```

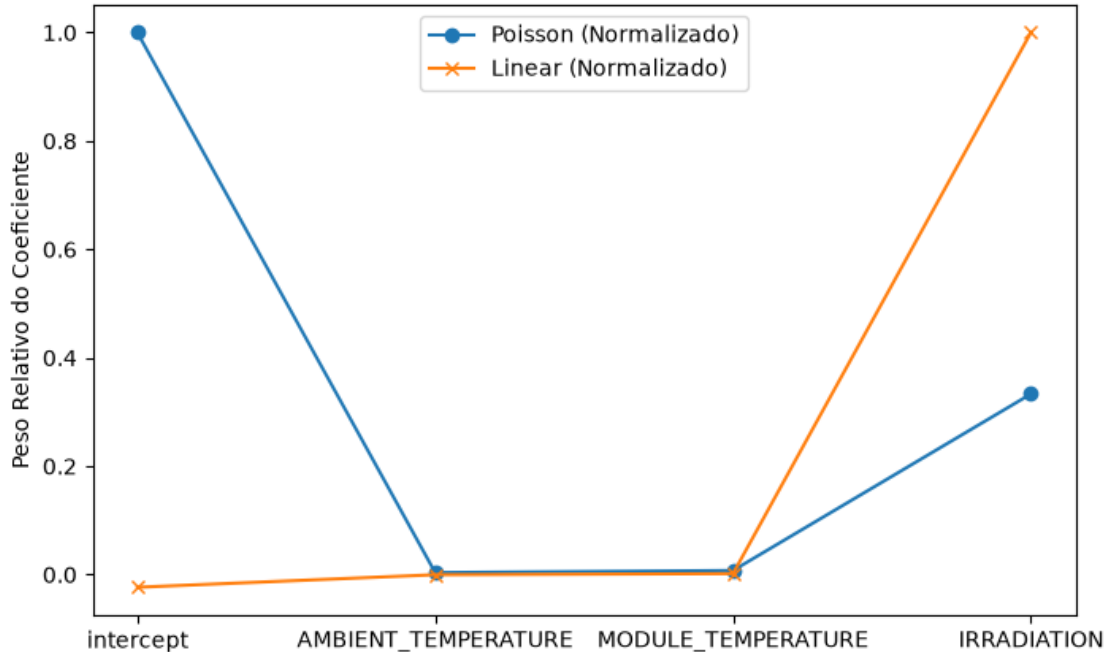
```
[20]:
```

	coef	std err	z	P> z
intercept	5.2458	0.001000	6097.218	0.0
AMBIENT_TEMPERATURE	0.0187	0.000049	384.099	0.0
MODULE_TEMPERATURE	0.0390	0.000029	1368.527	0.0
IRRADIATION	1.7477	0.001000	2035.135	0.0

Plota os coeficientes dos preditores (Temperaturas e Irradiação) para ilustrar e comparar a forma como o modelo Linear e o de Poisson interpretam a variação e o peso de cada fator na geração de energia.

```
[21]: coef_pois = pois_mod.params
coef_lin = lin_mod.params

import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(8, 5))
# Normalizando os coeficientes para ficarem na mesma escala visual de
# comparação no gráfico
ax.plot(coef_pois.index, coef_pois.values / max(abs(coef_pois.values)),
        marker='o', label='Poisson (Normalizado)')
ax.plot(coef_lin.index, coef_lin.values / max(abs(coef_lin.values)),
        marker='x', label='Linear (Normalizado)')
ax.set_ylabel('Peso Relativo do Coeficiente')
ax.legend();
```



O livro demonstra que o modelo original com todas as variáveis pode sofrer de *overfitting*. Ao reajustar o modelo utilizando apenas IRRADIATION e MODULE_TEMPERATURE (as variáveis com maior

poder preditivo no nosso caso), testamos um modelo mais simples. Também demonstra como prever instâncias específicas com dados arbitrários novos.

```
[22]: X_train_sub = X_train_drop[['IRRADIATION', 'MODULE_TEMPERATURE']]
X_test_sub = X_test_drop[['IRRADIATION', 'MODULE_TEMPERATURE']]

glm_sub = sm.GLM(y_train, X_train_sub, family=sm.families.Binomial())
results_sub = glm_sub.fit()
probs_sub = results_sub.predict(exog=X_test_sub)

labels_sub = np.array(['Negativo'] * len(X_test_sub))
labels_sub[probs_sub > 0.5] = 'Positivo'
print("Matriz de Confusão (Irradiação e Temp. Módulo):\n",
      ↪confusion_matrix(y_test_cat, labels_sub))

# Previsão para novos valores arbitrários (ex: alta irradiação/temp vs baixa
  ↪irradiação/temp)
import pandas as pd
newdata = pd.DataFrame({'IRRADIATION': [0.8, 0.0], 'MODULE_TEMPERATURE': [45.0,
  ↪22.0]})
print("\nPrevisões de probabilidade para novos instantes:\n", results_sub.
  ↪predict(newdata))
```

Matriz de Confusão (Irradiação e Temp. Módulo):

```
[[10357   17]
 [  330 9929]]
```

Previsões de probabilidade para novos instantes:

```
0    1.000000
1    0.029194
dtype: float64
```

O código original mostra como inspecionar as probabilidades geradas pelo modelo de Análise Discriminante Linear (LDA) e aplicar uma regra de corte mais rigorosa. Em vez do corte padrão de 50%, verificamos quantos registros o modelo previu como geração alta (Positivo) com mais de 90% de certeza.

```
[23]: lda_prob = lda.predict_proba(X_test_drop)
# np.where encontra o índice da classe 'Positivo' dentro do lda.classes_
pos_idx = np.where(lda.classes_ == 'Positivo')[0][0]
print("Instâncias com > 90% de probabilidade de geração Alta (Positivo):", np.
      ↪sum(lda_prob[:, pos_idx] > 0.9))
```

Instâncias com > 90% de probabilidade de geração Alta (Positivo): 6204

Demonstra como extrair os parâmetros internos aprendidos pelo modelo Naive Bayes, especificamente os *priors* das classes (probabilidade prévia de Positivo/Negativo) e as variâncias (var_) estimadas para cada variável dentro de cada classe.

```
[24]: print("Classes do Naive Bayes:", nb.classes_)
      print("Probabilidades prévias (Priors):", nb.class_prior_)
      print("Variâncias estimadas para cada feature nas respectivas classes:\n", nb.
            ↪var_)
```

```
Classes do Naive Bayes: ['Negativo' 'Positivo']
Probabilidades prévias (Priors): [0.49880559 0.50119441]
Variâncias estimadas para cada feature nas respectivas classes:
[[2.89729180e+00 4.90915093e+00 1.25621775e-03]
 [9.67532892e+00 1.13246218e+02 7.62039251e-02]]
```

Esta é a etapa de avaliação final do dataset contínuo. O código calcula e compara o Erro Quadrático Médio (MSE) gerado pelo modelo Linear contra o modelo de Poisson, validando numericamente qual modelo performa melhor na previsão da potência.

```
[25]: predict_lm = lin_mod.predict(X_test_cont)
      print("MSE Regressão Linear:", np.mean((predict_lm - y_test_cont)**2))

      predict_pois = pois_mod.predict(X_test_cont)
      print("MSE Regressão de Poisson:", np.mean((predict_pois - y_test_cont)**2))
```

```
MSE Regressão Linear: 312825.7343550514
MSE Regressão de Poisson: 3850097.6822358957
```